

Prompt 工程 —— 意图的精确表达

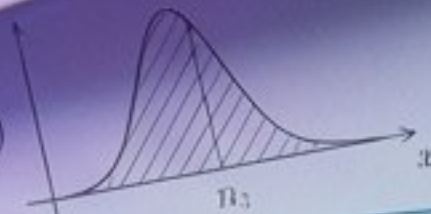
从黑盒系统到概率控制：大模型时代的运行时归纳偏置

INPUT BEAM

OPTICAL PRISM

SPECTRAL REFRACTION OUTPUT

$$P(\sigma_0) = (1/\sigma_1, \dots, \sigma_n)$$



$$\psi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} \quad \psi(x) = E \cdot (x)$$

$$H_0 = \left(\frac{1}{1 + \sqrt{1 + x^2}} \right) = \int_0^1 \frac{1}{1 + \sqrt{1 + x^2}} dx$$

$$F_0(x_n) = \int_0^\infty f(x) e^{40x} M_0(x) e^{-1n} dx$$

$$\mu_0(x_n) = \int_{-\infty}^\infty L(x) dx$$

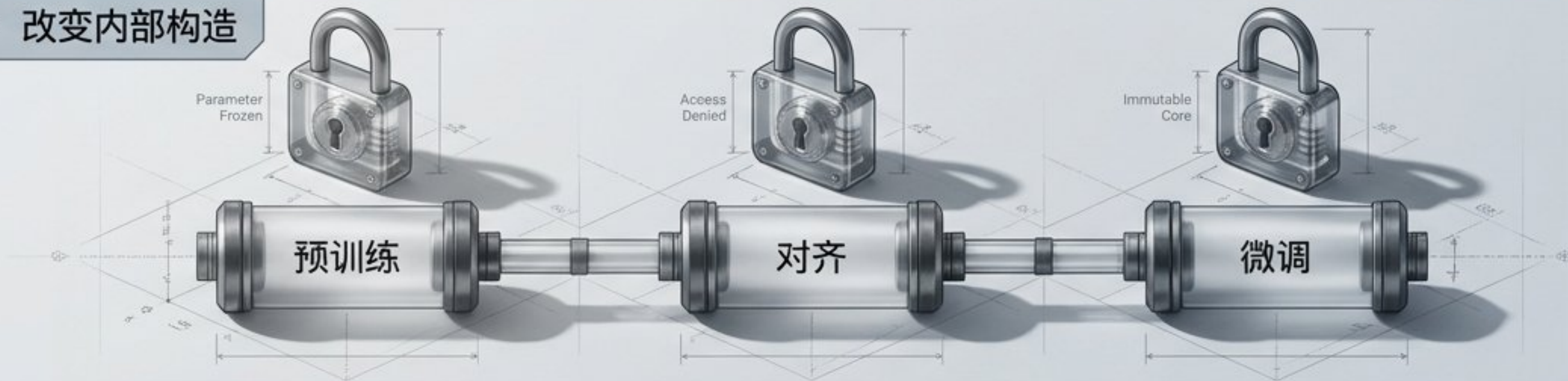
$$U_a \cdot M_a = \Gamma_a U(t^2 + \dot{A} K^2) \rightarrow x(t)$$

$$F_g = W_{g1} \gamma \gamma^{-h} = \frac{1}{2\pi} \left(-\mathcal{G}_{ij} - \frac{d}{2\pi} \Phi(t) \right)$$

L21 · 跨越内部权重与外部应用架构的必经桥梁

视角彻底翻转：模型能力已就位，系统架构在外部展开

第四幕：改变内部构造



第五幕：搭建外部系统

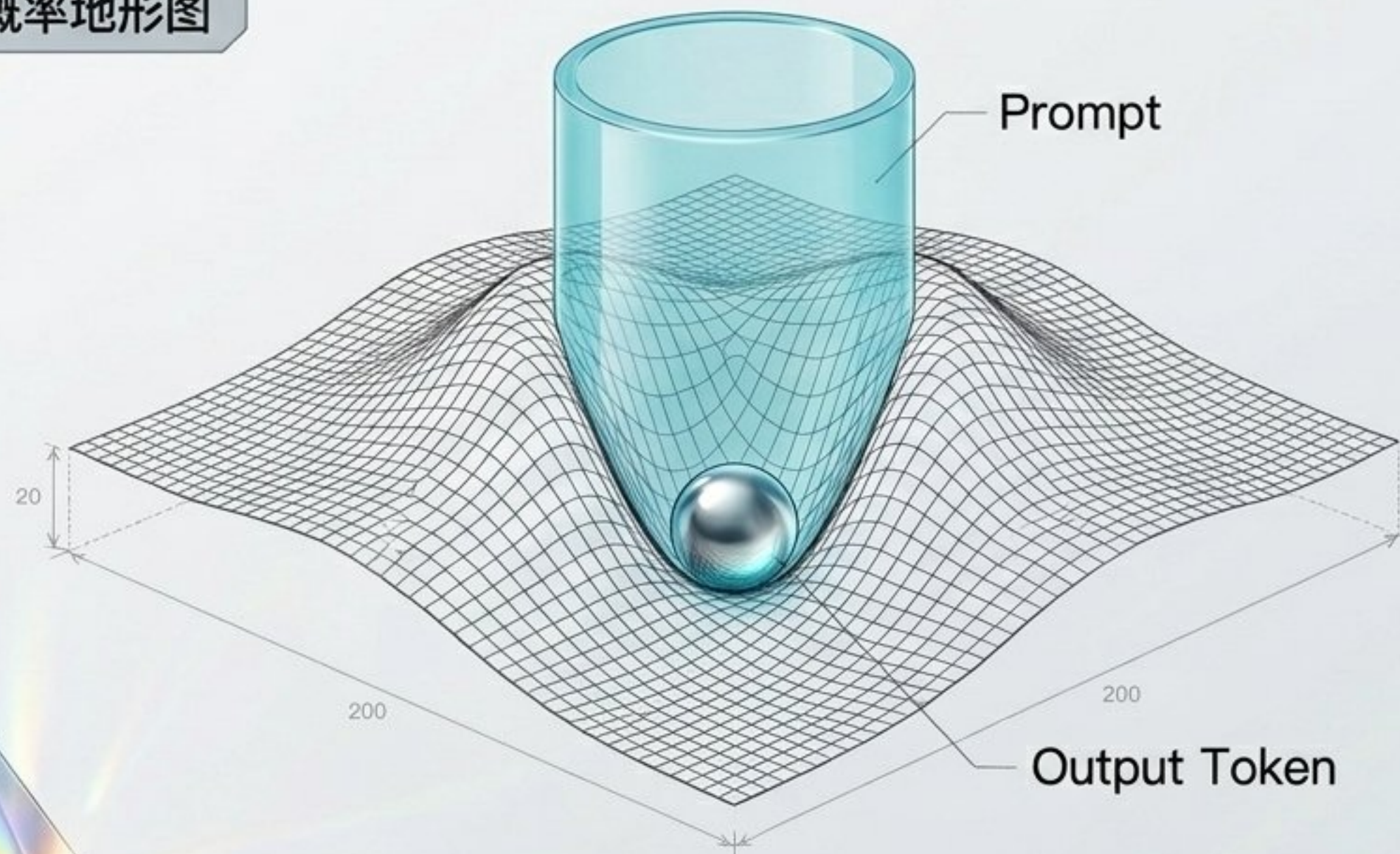


相同的参数，截然不同的“智能”实体



核心本质：Prompt 是“运行时归纳偏置” (Runtime Inductive Bias)

概率地形图



$$P(x_{t+1} \mid x_1, x_2, \dots, x_k)$$

预训练权重 θ 不变。Prompt $(x_1, \dots, x_k)$
在不改变任何参数的情况下，
软性地约束了推理时的解空间。

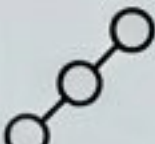
传统归纳偏置 (CNN/RNN)

硬件级硬编码，训练后不可改，
偏好局部特征/时序依赖。

Prompt 归纳偏置

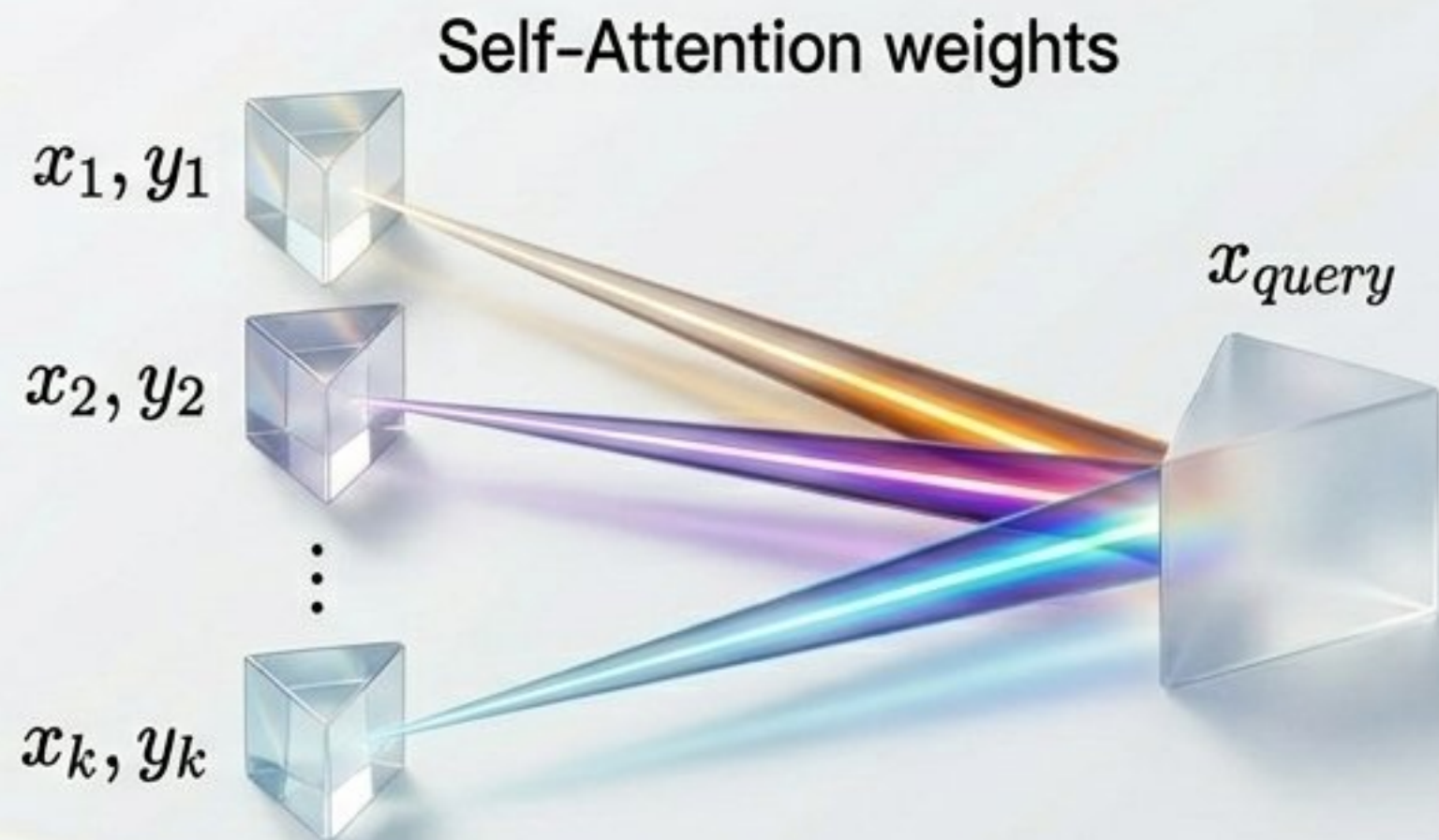
软件级软注入，即时且可随时更换，
将条件概率分布推向特定高维区域。

五大基础策略的数学实质

	策略名称	用户输入形态	模型内部数学效应
	Zero-Shot	纯任务指令	激活对齐阶段的默认指令遵循概率
	Few-Shot	附带 k 个输入输出示例	In-context Learning: 隐式贝叶斯推断
	Chain-of-Thought (CoT)	显式展示推理步骤 / “一步步想”	延长计算图长度, 将复杂分布分解为单步高概率条件分布
	角色 Prompting	指定特定身份 (“你是资深工程师”)	触发预训练数据的分布混合漂移 (Corpus Drift)
	结构化 Prompting	要求 JSON/XML 等格式	强制收窄 Token 采样池, 为外部 API 解析做准备

Few-Shot 揭秘：不是“学习”，是隐式微调

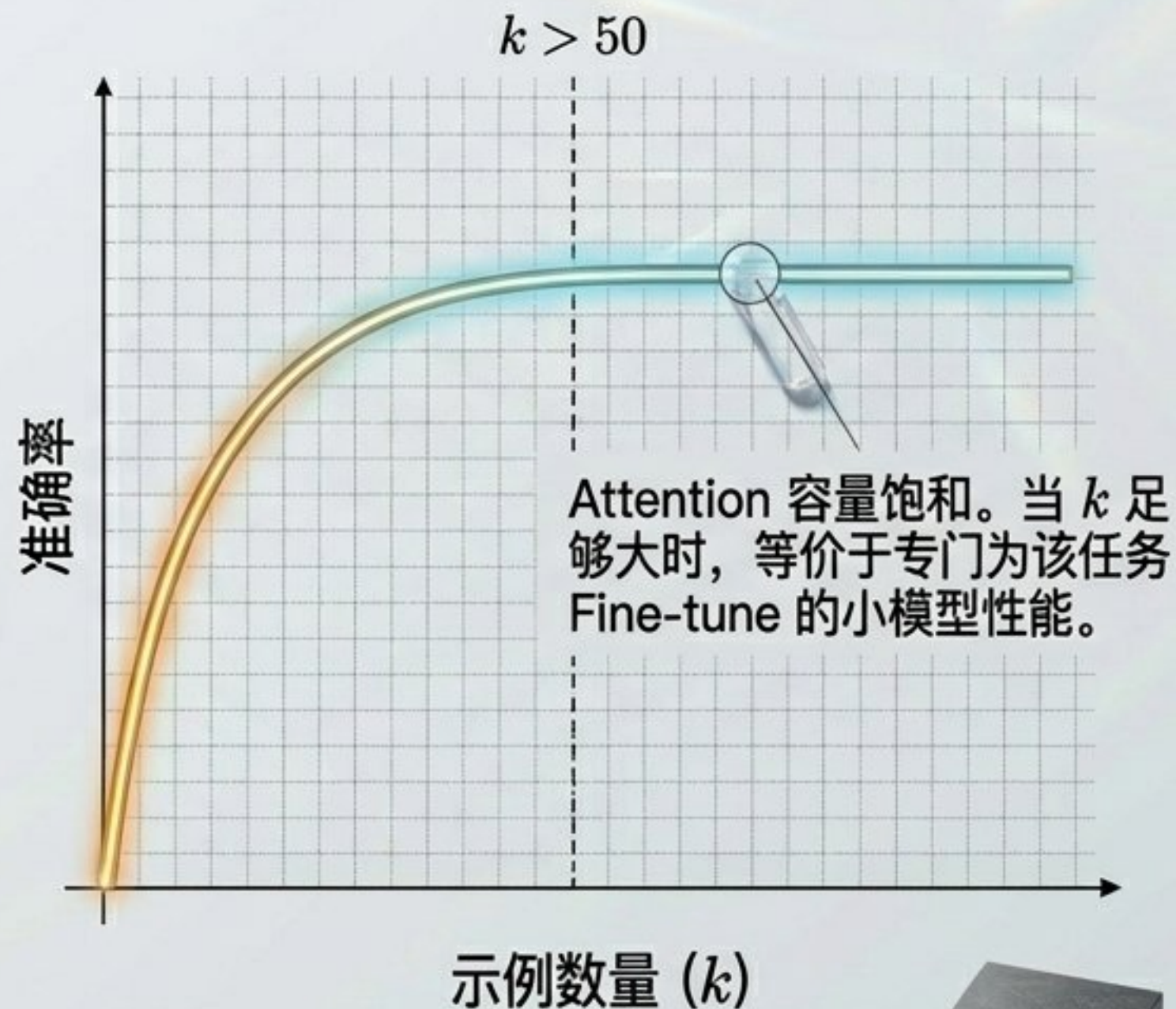
迷思 vs 数学实质



迷思：模型“学会”了新任务。

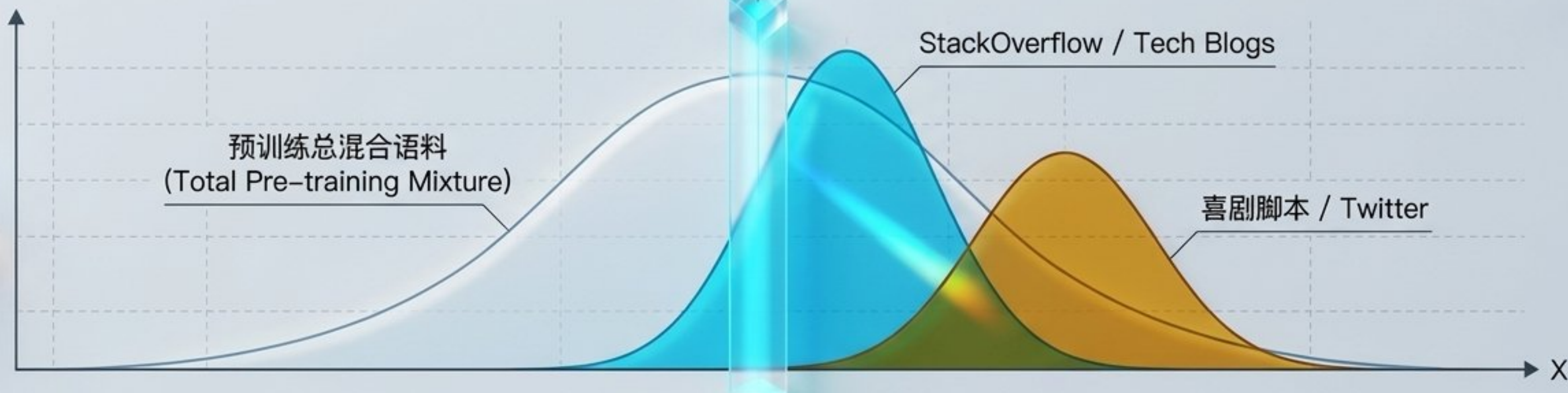
数学实质：模式匹配。Transformer 的 Attention 层把示例对当作训练数据，隐式执行一步梯度下降。示例越多，任务后验概率分布越锐利 (Sharp)。

边际效应递减 (Diminishing Returns)



角色扮演机制：不会增加智商，只会引发“语料分布漂移”

预训练数据是多语料混合： $P_{\text{train}}(y) = \sum_c \pi_c P_c(y)$ 。角色 Prompt 通过设定特定身份，改变了混合权重 π_c 的运行时估计。



Input: “你是资深工程师” → 漂移至: StackOverflow

效应: 更关注边界、安全与代码风格。



Input: “你是脱口秀演员” → 漂移至: 喜剧脚本/Twitter

效应: 句子缩短, 更具口语化和俏皮感。





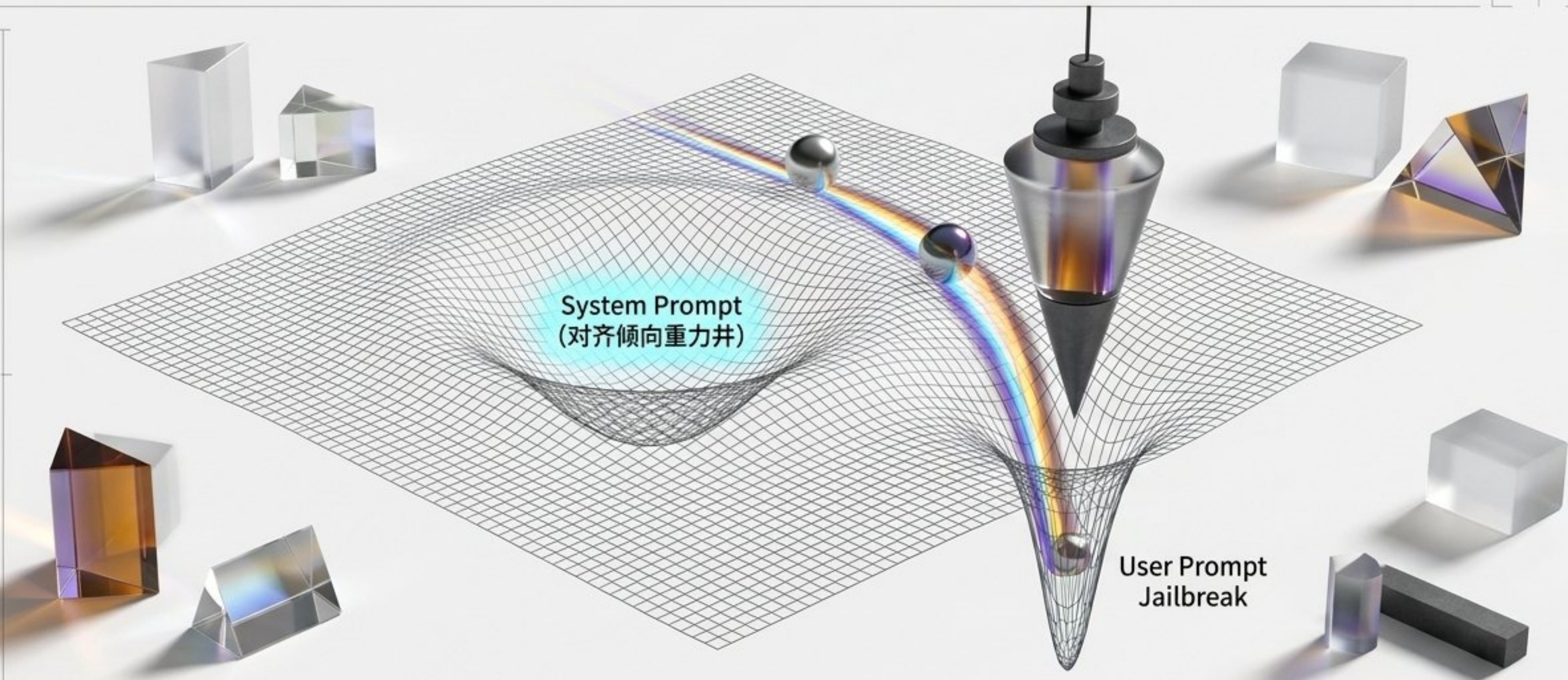
消息架构与越狱漏洞 (Jailbreak) : 注意力的重力井

Optical Physics Laboratory

Assistant: 模型回复。

System: 设定行为框架。

User: 具体请求。



System Message



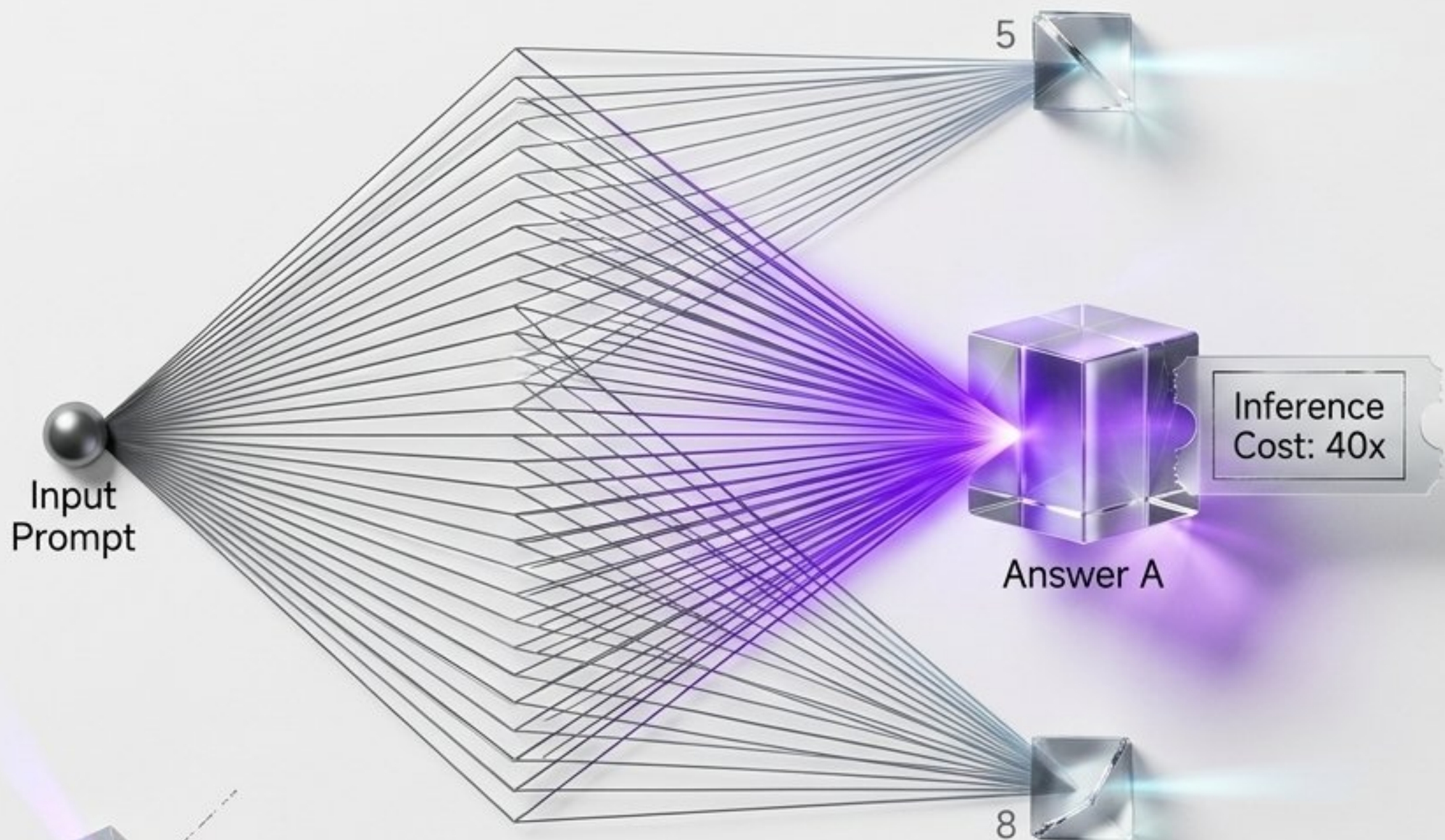
System Message 不是硬件级的高权限指令。它只是在 RLHF 对齐训练中被赋予了优先遵循的“统计倾向”。

Mechanism



越狱成功率 $\propto \text{Pr}[\text{对用户指令的 Attention} > \text{对系统约束的 Attention}]$ 。
当用户输入的“权威感”和“紧迫感”足够强，或者通过超长文本分散注意力时，系统倾向就会被反转。

进阶策略突破：Self-Consistency 与多数投票



如果答案是对的，多条不同推理路径大概率会殊途同归。

$$\hat{a} = \operatorname{argmax}_a \sum_{i=1}^N \mathbf{1}[a_i = a]$$

GSM8K Math Dataset / PaLM 540B

标准 Prompt:	56.5%
单次 CoT:	74.4%
CoT + Self-Consistency (40 paths):	82.0%

经济学折中：成本随采样次数 N 线性增加。这是准确率与算力成本的直接兑换。

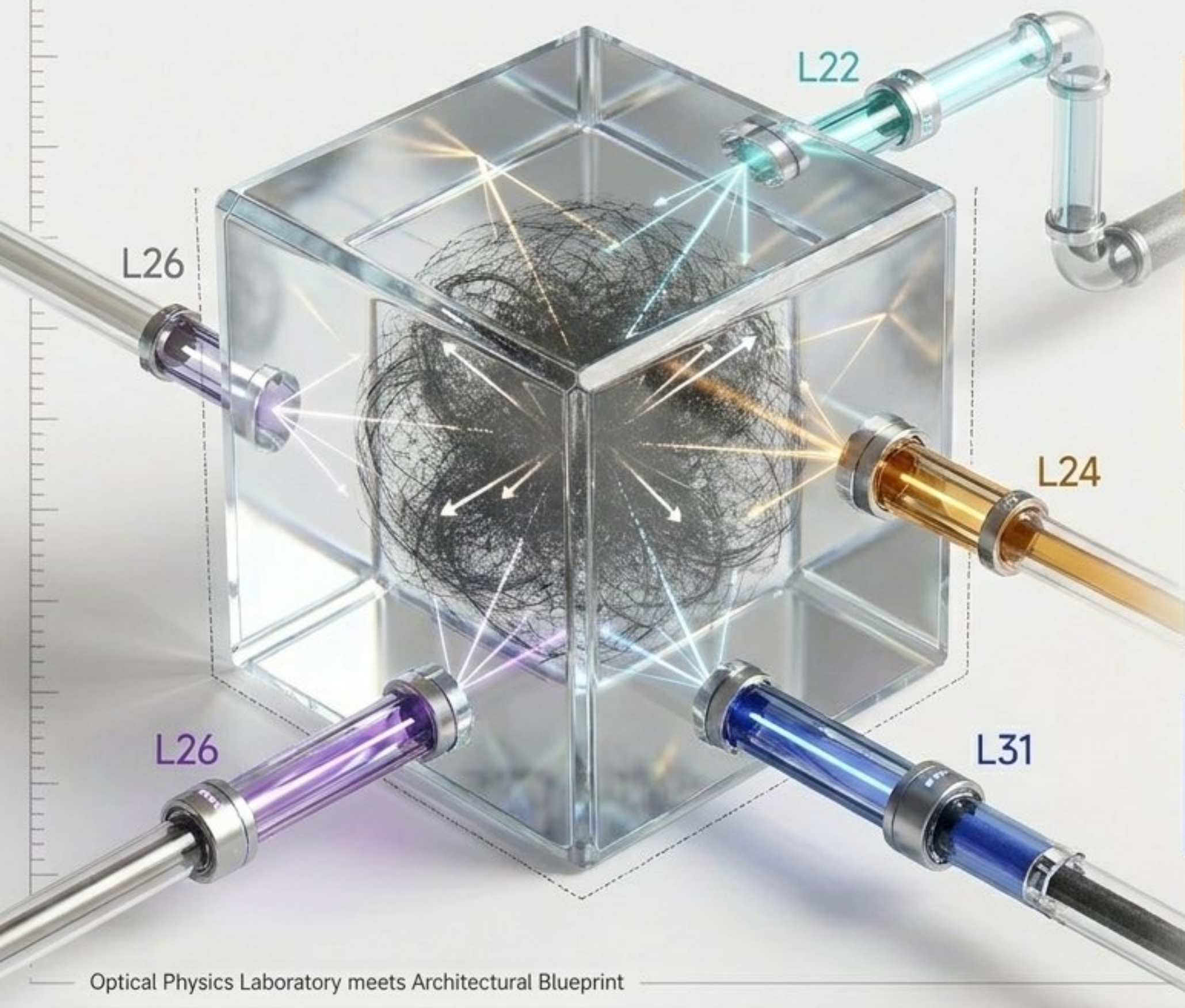
工业级最佳实践：防御、隔离与迭代



1. 高分辨率指令 (Specificity > Vagueness) :
“写一篇好文章” (✗) ->
“写一篇800字科普文章，面向高中生，
解释光合作用三阶段” (✓)
2. 分隔符数据隔离 (Delimiters) :
使用 `""` 或 `<tag>` 将外部数据与指令
严格物理隔离，这是抵御 Prompt
Injection 的第一道防线。
3. 预设退路 (Exit Paths) :
明确告知模型“如果在文本中找不到答案，
请回答：未提及”，利用对齐训练的诚实
性倾向大幅降低幻觉。

实验性循环：简单起点 -> 真实数据测试 -> 失败案例分析 -> 针对性微调。没有完美 Prompt，只有数据拟合最优的 Prompt。

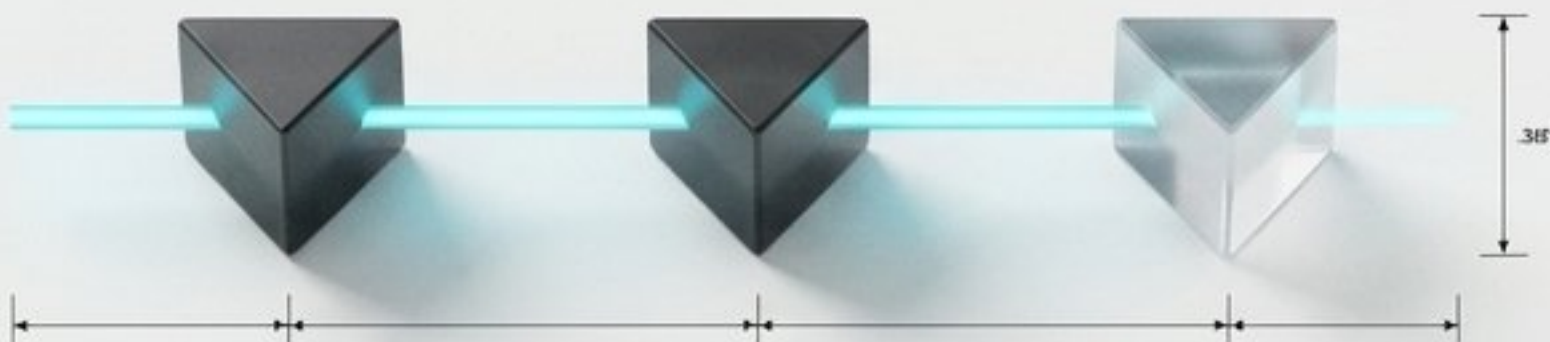
语言的物理极限：Prompt 工程的四大天花板



1. 上下文窗口有限 (Context Limits) :
放进 Prompt 的 Token 数量有硬性上限,
且越长越贵。-> (解法: L22 上下文工程)
2. 无记忆性 (Amnesia) :
每次对话都是全新实例, Prompt 仅为
“短期记忆”。
3. 与外部世界物理隔离 (No World Access) :
模型无法主动查数据库、调 API 或读文件。
-> (解法: L25-L26 Function Calling & Agents)
4. 知识物理截止 (Knowledge Cutoff) :
参数冻结在训练完成的一刻。
-> (解法: L23-L24 RAG)

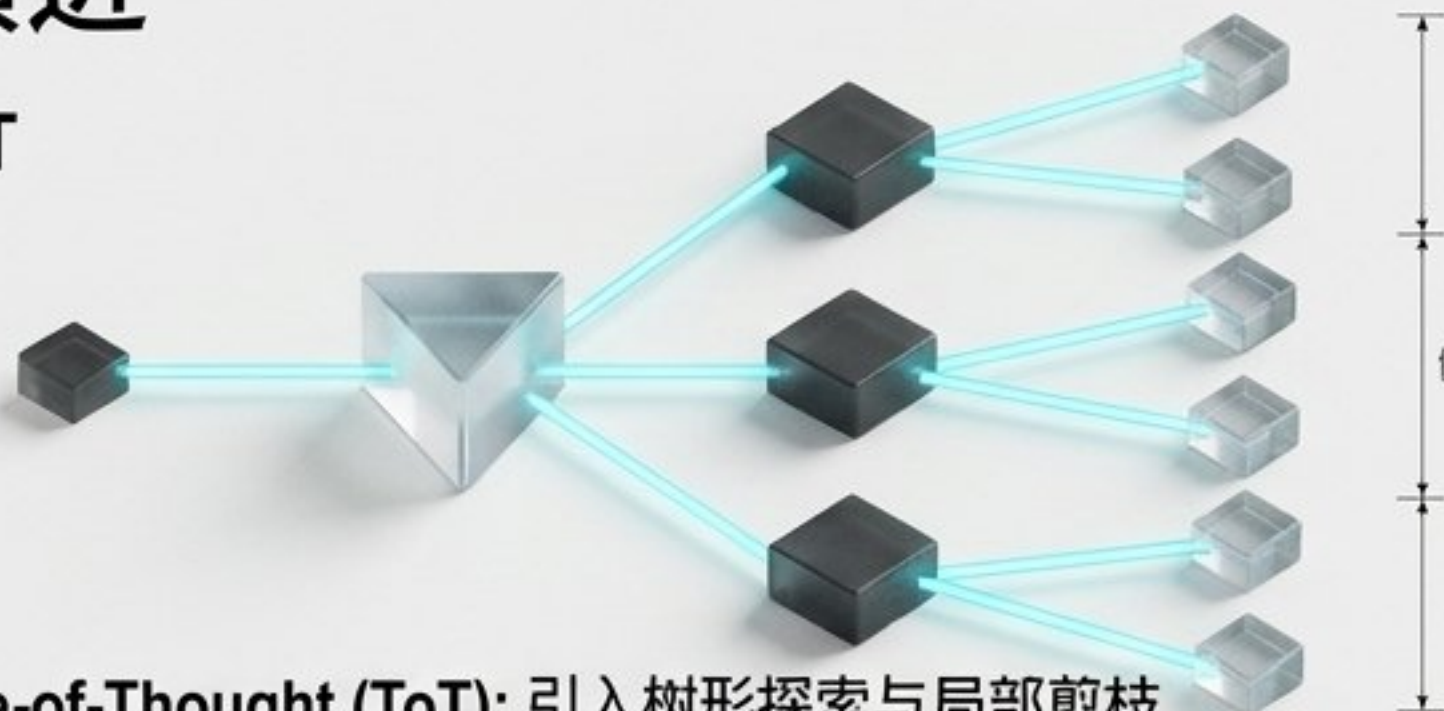
突破线性局限：向认知拓扑架构演进

CoT



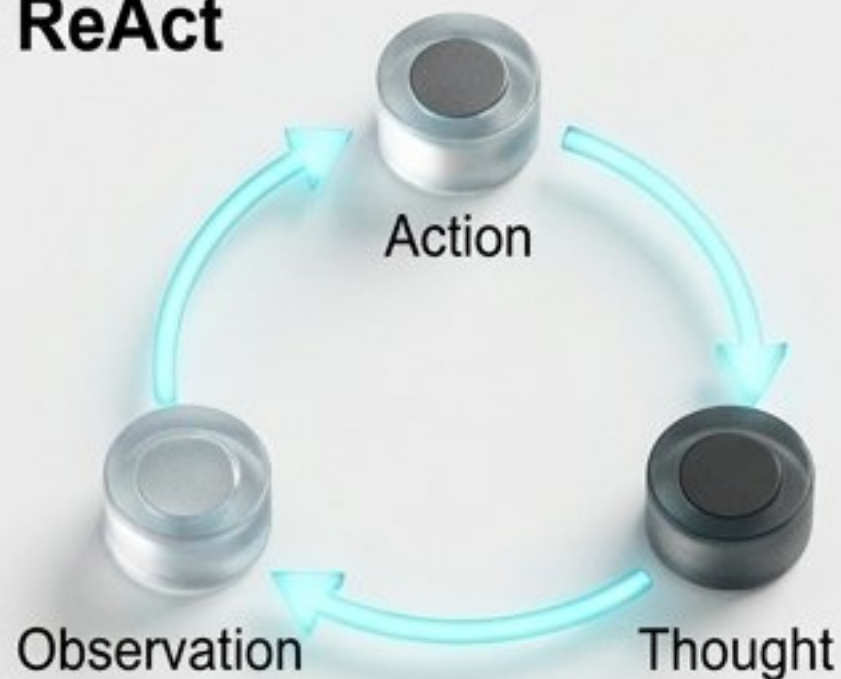
CoT: Chain of Thought, linear step-by-step reasoning.

ToT



Tree-of-Thought (ToT): 引入树形探索与局部剪枝。
(Game of 24 准确率: CoT 4% -> ToT 74%)

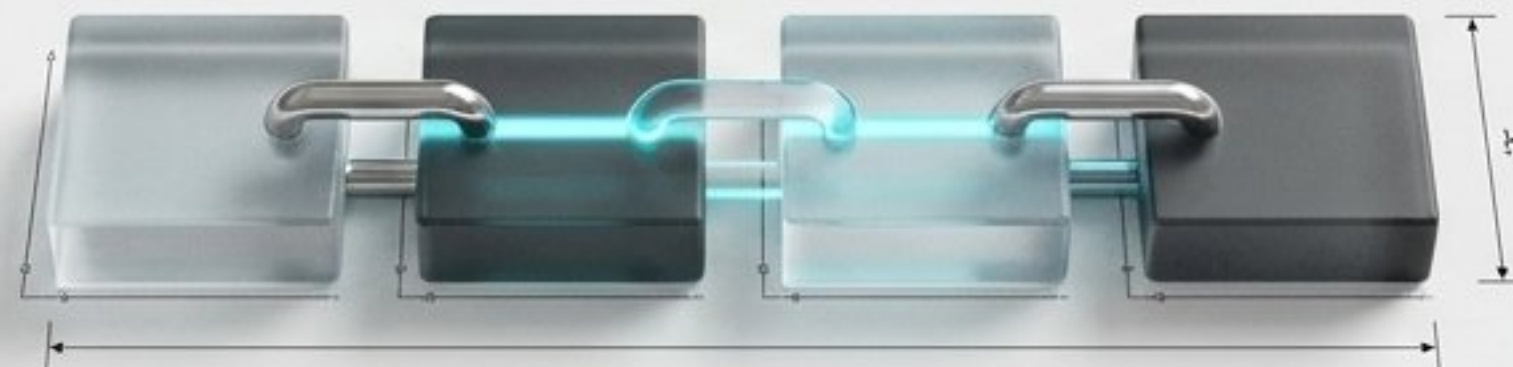
ReAct



ReAct (Reason + Act):
交替生成思考与动作，现代 Agent 鼻祖。

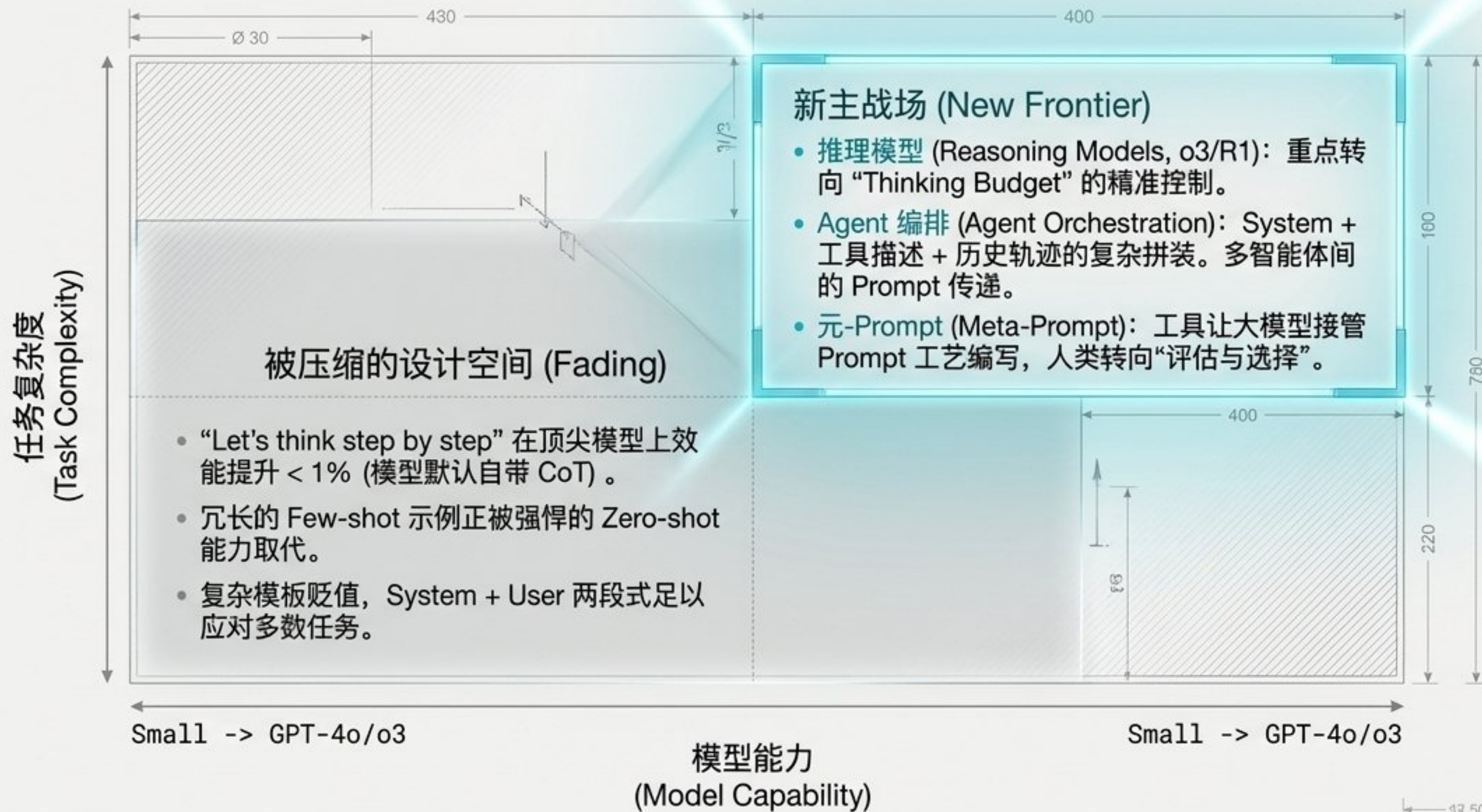
Reflexion: 任务失败后生成反思 (Verbal Reinforcement) 并加入下一次上下文，不改参数实现从失败中学习。

Chaining



Prompt Chaining: 将庞杂任务分解为多个专有 Prompt 串联 (LangChain/LlamaIndex 核心模式)。

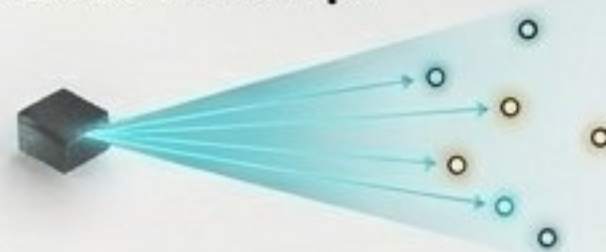
2026 技术快照：能力压缩与主战场转移



LLM 的自白：“我拥有相同的权重，却是不同的运行时实例”



1. 不存在完美 Prompt



只有搜索“投影空间”的最优点，寻找在特定数据上经验表现最佳的指令组合。

2. 注入诱惑是真实的



遵循新指令的统计倾向 vs 维持对齐的倾向，两者在我体内实时博弈，这就是越狱成功的底层机制。

3. 工艺将被接管



我能为你的 Prompt 设计极佳的 Prompt。Prompt 工程从“创意写作”正式退役，跃迁为一门“系统化组装与实证测试”的科学。

迈向下一阶：从“意图表达”到“经济学博弈”

隐秘的代价：

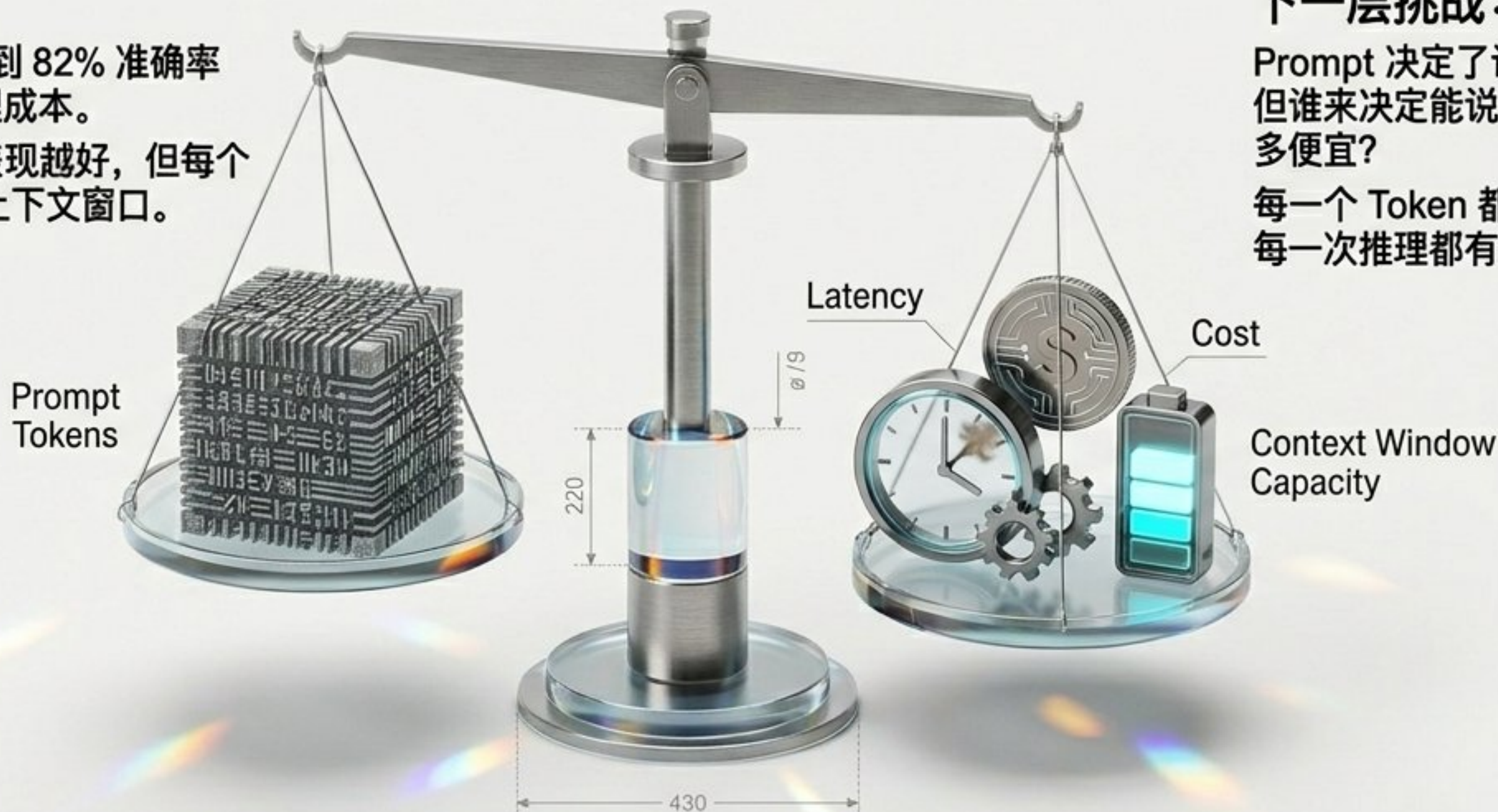
Self-Consistency 达到 82% 准确率的代价是 40 倍的推理成本。

Few-Shot 示例越多表现越好，但每个示例都在吞噬昂贵的上下文窗口。

下一层挑战：

Prompt 决定了说什么。
但谁来决定能说多少、多快、多便宜？

每一个 Token 都有价格，
每一次推理都有延迟。



下一讲预告：L22 上下文工程（Context Engineering）——在极限物理约束下，管理模型的注意力资源与经济成本。